

TIPE: Modélisation des résultats des matchs de football

Application à la Premier League

OUNZAR Aymane - n° 18290

LM6E

2024/2023

- 1 Introduction :
 - Modèle de base :
- 2 Détermination des paramètres :
 - Maximum de vraisemblance
 - Maximum de vraisemblance pondéré
- 3 Dépendance :
 - Y a-t-il dépendance entre les variables aléatoires?
 - Les copules
- 4 Les modèles :
 - Premières vérifications
- 5 Comparaison
 - Critère d'Akaike
 - Pariage
 - Résultats
- 6 Commentaires :
- 7 Annexe



Modèle de base :

Détails du modèle (Modèle 0) :

- Deux équipes H et A

Modèle de base :

Détails du modèle (Modèle 0) :

- Deux équipes H et A
- L'équipe H joue sur son terrain et A est visiteuse.

Modèle de base :

Détails du modèle (Modèle 0) :

- Deux équipes H et A
- L'équipe H joue sur son terrain et A est visiteuse.
- $X_H \sim P(\alpha_H \beta_A \gamma)$ et $X_A \sim P(\alpha_A \beta_H)$ où γ représente l'avantage du terrain

Modèle de base :

Détails du modèle (Modèle 0) :

- Deux équipes H et A
- L'équipe H joue sur son terrain et A est visiteuse.
- $X_H \sim P(\alpha_H \beta_A \gamma)$ et $X_A \sim P(\alpha_A \beta_H)$ où γ représente l'avantage du terrain
- X_H et X_A sont considérés indépendants.

Modèle de base :

Détails du modèle (Modèle 0) :

- Deux équipes H et A
- L'équipe H joue sur son terrain et A est visiteuse.
- $X_H \sim P(\alpha_H \beta_A \gamma)$ et $X_A \sim P(\alpha_A \beta_H)$ où γ représente l'avantage du terrain
- X_H et X_A sont considérés indépendants.
- les coefficients $\alpha_H, \beta_A, \alpha_A, \beta_H$ change avec le temps .

Pourquoi Poisson :

Approximation :

Chaque fois une équipe a le ballon, elle va essayer de marquer un but.

On note par p la probabilité que le but va être marqué

Puisqu'une équipe peut avoir plusieurs occasions pendant un match pour marquer, donc le nombre de buts marqués par l'équipe est binomial.

Pourquoi Poisson :

Approximation :

Chaque fois une équipe a le ballon, elle va essayer de marquer un but.
On note par p la probabilité que le but va être marqué
Puisqu'une équipe peut avoir plusieurs occasions pendant un match pour marquer, donc le nombre de buts marqués par l'équipe est binomial.

Puisque p est très petit et le nombre de fois où l'équipe a le ballon est grand. On peut approcher la loi Binomiale par la loi de poisson.

Maximum de vraisemblance :

On dispose d'un échantillon de N matchs $(x_1, y_1), \dots, (x_N, y_N)$ où x_k est le nombre de buts marqués par l'équipe à domicile dans un match k et y_k est le nombre de buts marqués par l'équipe visiteuse j_k pendant le match k .

Estimateur de vraisemblance :

Il est défini par :

$$\mathcal{L}(\alpha, \beta, \gamma) = \prod_{k \in \mathcal{A}} P(X_H = x_k, X_A = y_k)$$

où $\mathcal{A}$ est l'ensemble des matchs

Maximum de vraisemblance :

On cherche les valeurs des paramètres qui maximise l'estimateur de vraisemblance, $\mathcal{L}(\alpha, \beta, \gamma)$ ce qui est équivalent à maximiser :

$$\ell(\alpha, \beta, \gamma) = \ln(\mathcal{L}(\alpha, \beta, \gamma))$$

Maximum de vraisemblance pondéré :

La fonction du maximum de vraisemblance pondérée est donc donnée par :

$$\tilde{\ell}(\alpha, \beta, \gamma, \tau) = \sum_{k \in \mathcal{A}_t} \ln(P(X_H = x_k, X_A = y_k)) \Phi(t - t_k)$$

avec :

- $\mathcal{A}_t$ est l'ensemble des matchs ayant lieu avant le jour t
- t_k est le jour du match k
- t est le jour de détermination des paramètres

Choix de Φ :

On peut supposer que la décroissance est exponentielle :

$$\Phi : t \longmapsto e^{-\theta(t-t_k)}$$

Détermination de θ :

On ne peut pas déterminer la valeur de θ en utilisation la méthode du maximum de vraisemblance car :

$$\lim_{\theta \rightarrow +\infty} \tilde{\ell}(\alpha, \beta, \gamma, \tau, \theta) = 0$$

quelles que soient les données qu'on utilise.

Ordre de grandeur de θ :

On va utiliser les données des années 2021/2022 et 2022/2023 lors de l'estimation des paramètres du modèle.

La durée caractéristique τ est de l'ordre de grandeur de

$$365 \times 2 = 730 \text{ jours}$$

Alors l'ordre de grandeur de $\theta = \frac{1}{\tau}$ est : 10^{-3}

Ordre de grandeur de θ :

On va utiliser les données des années 2021/2022 et 2022/2023 lors de l'estimation des paramètres du modèle.

La durée caractéristique τ est de l'ordre de grandeur de

$$365 \times 2 = 730 \text{ jours}$$

Alors l'ordre de grandeur de $\theta = \frac{1}{\tau}$ est : 10^{-3}

$$\theta \in \{0.001, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.008, 0.009\}$$

Choix de θ :

Pour chaque θ on détermine les valeurs de α, β, γ et τ qui maximisent $\tilde{\ell}(\alpha, \beta, \gamma, \tau, \theta)$. On garde la valeur de θ pour laquelle :

$$S(\theta) = \sum_{k=1}^N (\delta_k^H \log p_k^H + \delta_k^A \log p_k^A + \delta_k^D \log p_k^D + \gamma_k^{O2.5} \log p_k^{O2.5} + \gamma_k^{U2.5} \log p_k^{U2.5})$$

est maximale.

Dépendance :

Soit $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ un ensemble d'observations des variables jointes X et Y tel que les valeurs des (x_i) et (y_i) sont uniques.

Définition :

On dit qu'un pair (x_i, y_i) et (x_j, y_j) est **concordant** si $x_i < x_j$ et $y_i < y_j$ ou si $x_i > x_j$ et $y_i > y_j$

Définition :

On dit qu'un pair (x_i, y_i) et (x_j, y_j) est **discordant** si $x_i < x_j$ et $y_i > y_j$ ou si $x_i > x_j$ et $y_i < y_j$

Tau de kendall:

Le Tau de kendall mesure la corrélation de rang entre deux variables. Il est défini ainsi :

Le tau de Kendall est alors défini comme :

$$\tau_d = \frac{(\text{nombre de paires concordantes}) - (\text{nombre de paires discordantes})}{\frac{1}{2} \cdot n \cdot (n - 1)}$$

Tau de kendall:

Le Tau de kendall mesure la corrélation de rang entre deux variables. Il est défini ainsi :

Le tau de Kendall est alors défini comme :

$$\tau_d = \frac{(\text{nombre de paires concordantes}) - (\text{nombre de paires discordantes})}{\frac{1}{2} \cdot n \cdot (n - 1)}$$

D'après les données des années 2011 à 2024, j'ai estimé la valeur

$$\tau_d = -0.0227$$

Définition :

Soit (X, Y) un vecteur aléatoire à valeurs dans $\mathbb{R}^2$. La loi de (X, Y) est caractérisée par :

$$\forall (x, y) \in \mathbb{R}^2, F(x, y) = P(X \leq x, Y \leq y).$$

F est la fonction de répartition bivariable de (X, Y) . Lois marginales de (X, Y) sont décrites par :

$$F_1(x) = P(X \leq x) = \lim_{y \rightarrow +\infty} F(x, y) = F(x, +\infty),$$

$$F_2(y) = P(Y \leq y) = \lim_{x \rightarrow +\infty} F(x, y) = F(+\infty, y).$$

Définition

Une copule bivariable est une fonction $C : \mathbb{I}^2 \rightarrow \mathbb{I}$ telle que :

- pour tout $u \in \mathbb{I}$, $C(u, 0) = C(0, u) = 0$;
- pour tout $u \in \mathbb{I}$, $C(u, 1) = C(1, u) = u$;
- pour tout $u_1, u_2, v_1, v_2 \in \mathbb{I}$ tels que $u_1 \leq u_2$ et $v_1 \leq v_2$, on a :

$$C(u_2, v_2) - C(u_2, v_1) - C(u_1, v_2) + C(u_1, v_1) \geq 0.$$

Théorème de Sklar (Admis) :

- 1 Si C est une copule, si F_1 et F_2 sont deux fonctions de répartition, alors $F(x, y) = C(F_1(x), F_2(y))$ est une fonction de répartition bivariée, ayant F_1 et F_2 pour marginales.
- 2 Soit F une fonction de répartition bivariée de marginales F_1 et F_2 . Il existe une copule C telle que pour tout $(x, y) \in \mathbb{R}^2$:

$$F(x, y) = C(F_1(x), F_2(y)).$$

En 2009, Ian McHale et Phil Scarf ont trouvé que la structure de dépendance entre Y_H et Y_A peut être modélisée par la copule de Frank.

Copule de Frank :

$$C(u, v)_\tau = -\frac{1}{\tau} \ln \left(1 + \frac{(e^{-\tau u} - 1)(e^{-\tau v} - 1)}{e^{-\tau} - 1} \right)$$

- On note F_H la fonction de répartition de la loi marginale Y_H .
- On note F_A la fonction de répartition de la loi marginale Y_A .
- On note F la fonction de répartition bi-dimensionnelle du couple (Y_H, Y_A) .

- D'après le théorème de Sklar:

$$F(y_H, y_A) = C_\tau(F_H(y_H), F_A(y_A)) \\ = \frac{-1}{\tau} \ln \left(1 + \frac{(e^{-\tau F_H(y_H)} - 1)(e^{-\tau F_A(y_A)} - 1)}{e^{-\tau} - 1} \right)$$

- Dans ce cas, la probabilité que H marque y_H buts et que A marque y_A buts est :

$$P_\tau(Y_H = y_H, Y_A = y_A) = C_\tau(F_H(y_H), F_A(y_A)) - \\ C_\tau(F_H(y_H - 1), F_A(y_A)) - \\ C_\tau(F_H(y_H), F_A(y_A - 1)) + \\ C_\tau(F_H(y_H - 1), F_A(y_A - 1))$$

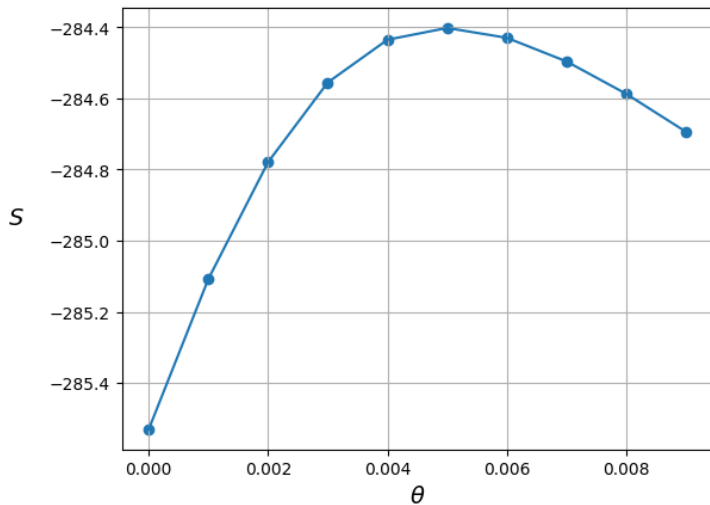
Échantillon :

On va appliquer le modèle sur les matchs des 14 **dernières semaines**, les **premiers 24 semaines** seront utilisées pour l'entraînement du modèle.

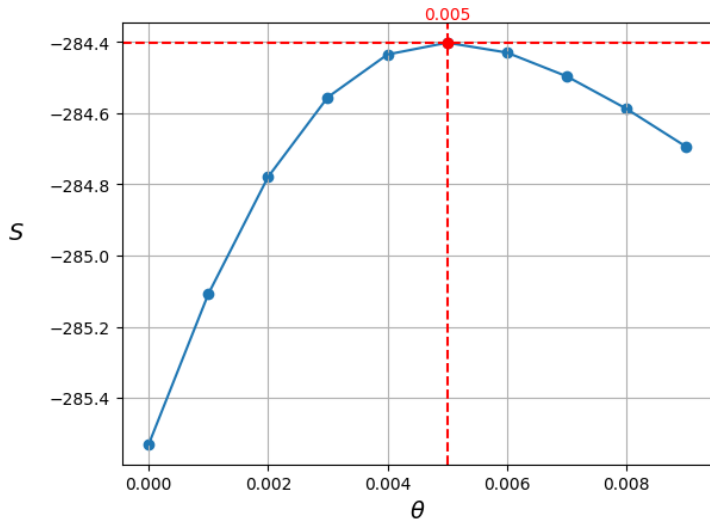
Les modèles :

	Estimateur de vraisemblance pondéré :	Copule :
Modèle 0	NON	NON
Modèle 1	OUI	NON
Modèle 2	OUI	OUI

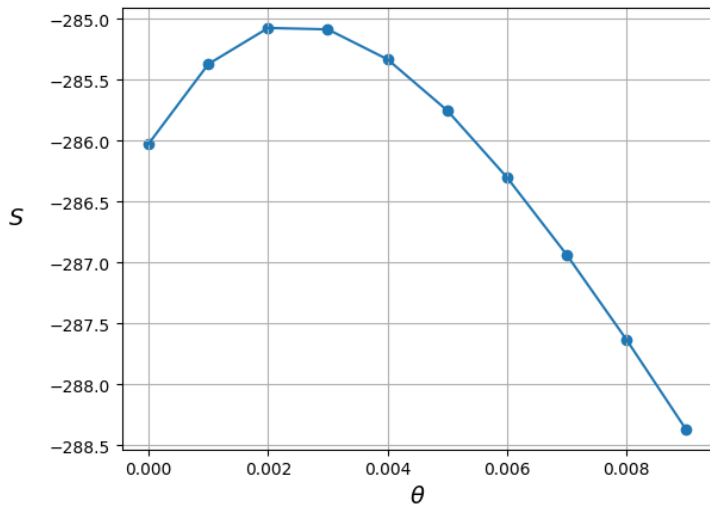
Valeur de θ modèle 1 :



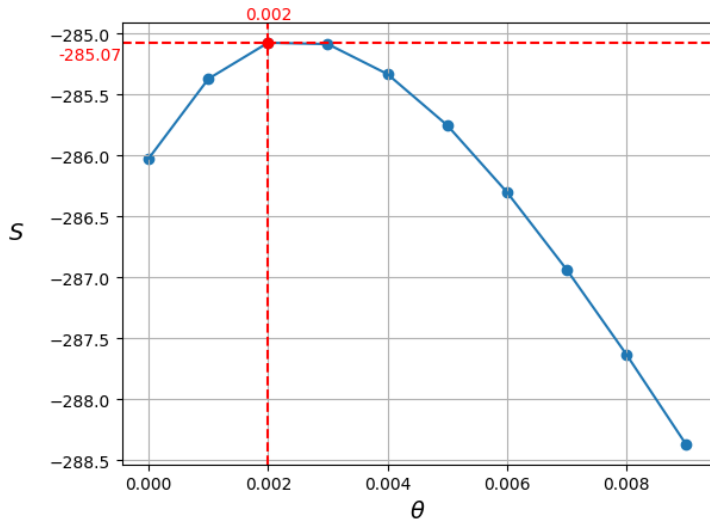
Valeur de θ modèle 1 :



Valeur de θ modèle 2



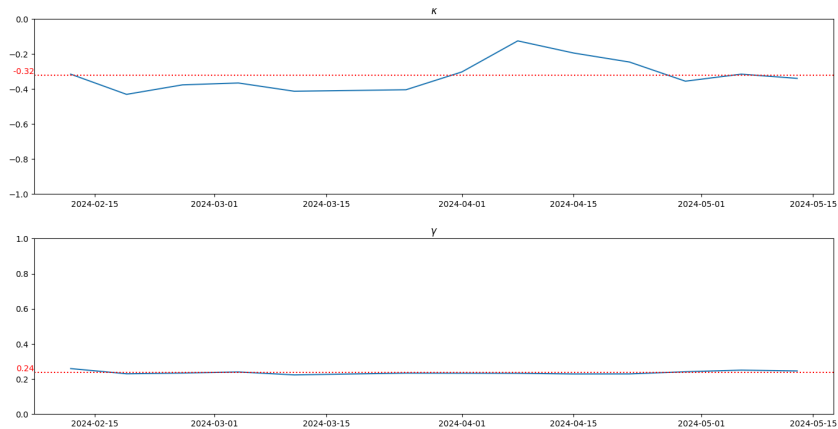
Valeur de θ modèle 2



Observations :

Équipe	α	Classement	$-\beta$	Classement
Arsenal	1.4365	2	1.1458	2
Aston Villa	1.1698	6	0.8325	5
Bournemouth	0.8327	14	0.5657	17
Brentford	0.9814	10	0.7428	13
Brighton	1.0973	7	0.7566	11
Burnley	0.6509	18	0.5255	18
Chelsea	1.0640	8	0.7843	8
Crystal Palace	0.7585	17	0.7459	12
Everton	0.6315	19	0.8010	6
Fulham	0.9480	13	0.7588	10
Liverpool	1.3674	3	1.1350	3
Luton Town	0.9813	11	0.4111	19
Manchester City	1.4656	1	1.2374	1
Manchester Utd	1.0160	9	0.8468	4
Newcastle Utd	1.2834	4	0.7945	7
Nott'ham Forest	0.7849	15	0.6090	16
Sheffield Utd	0.5668	20	0.2007	20
Tottenham	1.2370	5	0.7368	14
West Ham	0.9561	12	0.6471	15
Wolves	0.7709	16	0.7717	9

Observations :



Observations :

Proposition (Admise) :

Si τ_d est Tau de kendall et

$$\tau_d = 1 - 4\tau^{-1}(1 - \mathbf{D}_1(\tau))$$

où : $\mathbf{D}_n$ est la fonction de Debye définie par :

$$D_n(x) = \frac{n}{x^n} \int_0^x \frac{t^n}{e^t - 1} dt$$

En moyenne $\tau = -0.32$ donc $\tau_d = -0.035$

Définition :

Le critère d'information d'Akaike :

Le critère d'information d'Akaike s'écrit comme suit :

$$AIC = 2k - 2\ln(L)$$

où k est le nombre de paramètres à estimer du modèle et L est le maximum de la fonction de vraisemblance du modèle. On choisit alors le modèle avec le critère d'information d'Akaike le plus faible.

Résultats :

Modèle	Paramètres :	ℓ	AIC
Modèle 0	41	-441.120	964.240
Modèle 1	42	-438.110	960.22
Modèle 2	43	-438.833	963.666

Résultats :

Modèle	Paramètres :	ℓ	AIC
Modèle 0	41	-441.120	964.240
Modèle 1	42	-438.110	960.22
Modèle 2	43	-438.833	963.666

Modèle	Match gagné par équipe à domicile	match null	match gagné par équipe visiteuse	Total
Modèle 0	55/65	0/34	29/44	84/143
Modèle 1	49/65	0/34	34/44	83/143
Modèle 2	53/65	0/34	30/44	83/143

Résultats :

Modèle	Plus que 2.5 buts	Moins que 2.5 buts	Total
Modèle 0	71/92	15/51	86/143
Modèle 1	76/92	12/51	88/143
Modèle 2	81/92	11/51	92/143

- Marché 1X2
- Marché $O/U2.5$

Pour chaque match, on estime les paramètres des équipes, on calcule les probabilités et on décide si on va parier ou non

Critère de pariage :

On ne parie que lorsque :

$$E(A) = P(A)/b - 1 > t$$

où :

- A est un événement
- $P(A)$ est la probabilité de l'événement A
- b est la cote nette reçue sur le pari

$$\frac{1}{b+1}$$

peut être interprétée comme la probabilité de l'événement A estimée par le 'Bookmaker'.

Démarche :

- Détermination de la probabilité des événements par notre modèle

Démarche :

- Détermination de la probabilité des événements par notre modèle
- Calcul de la quantité $E(A)$

Démarche :

- Détermination de la probabilité des événements par notre modèle
- Calcul de la quantité $E(A)$
- Si $E(A) > t$ on parie avec la fraction f de nos fonds

Démarche :

- Détermination de la probabilité des événements par notre modèle
- Calcul de la quantité $E(A)$
- Si $E(A) > t$ on parie avec la fraction f de nos fonds
- Sinon on ne parie pas

Démarche :

- Détermination de la probabilité des événements par notre modèle
- Calcul de la quantité $E(A)$
- Si $E(A) > t$ on parie avec la fraction f de nos fonds
- Sinon on ne parie pas
- on utilise les 190 premiers matchs pour trouver une valeur pour t

Démarche :

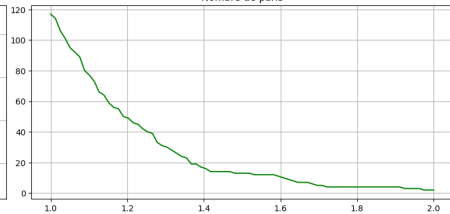
- Détermination de la probabilité des événements par notre modèle
- Calcul de la quantité $E(A)$
- Si $E(A) > t$ on parie avec la fraction f de nos fonds
- Sinon on ne parie pas
- on utilise les 190 premiers matchs pour trouver une valeur pour t
- on teste notre modèle sur les autres 190 matchs

Modèle 0 :

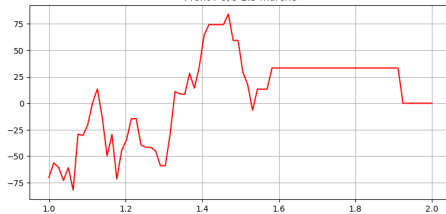
Profit : 1X2 marché



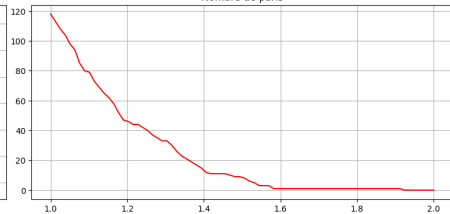
Nombre de paris



Profit : O/U 2.5 marché

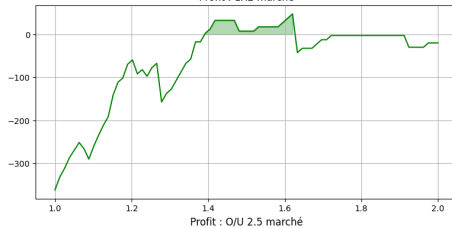


Nombre de paris

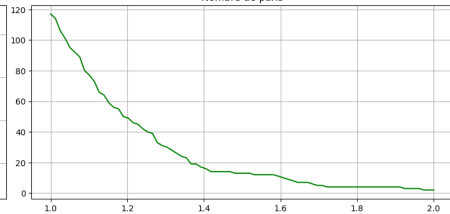


Modèle 0 :

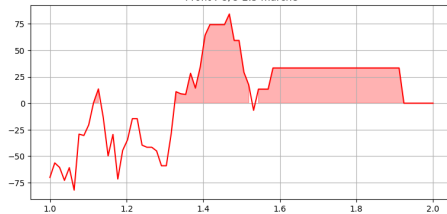
Profit : 1X2 marché



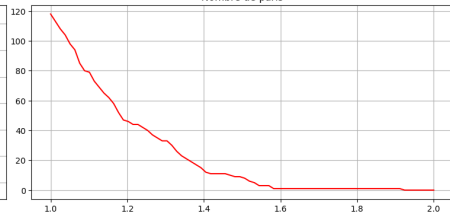
Nombre de paris



Profit : O/U 2.5 marché



Nombre de paris

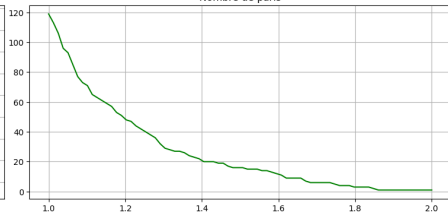


Modèle 1 :

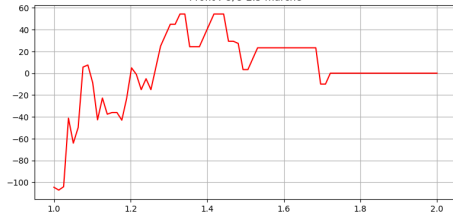
Profit : 1X2 marché



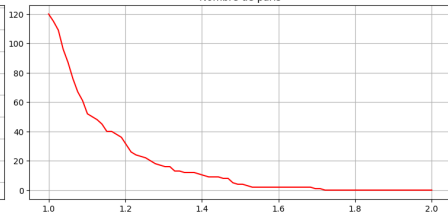
Nombre de paris



Profit : O/U 2.5 marché

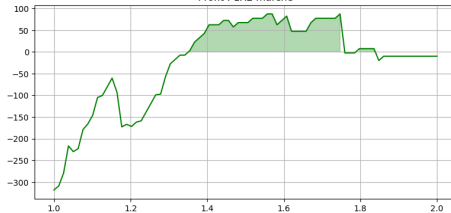


Nombre de paris

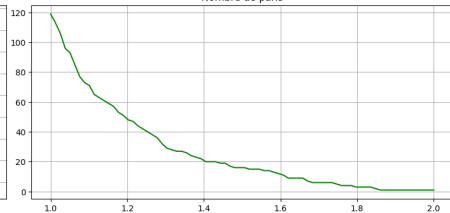


Modèle 1 :

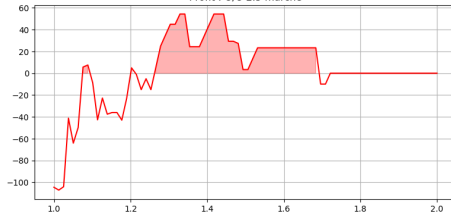
Profit : 1X2 marché



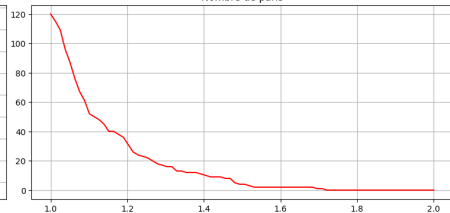
Nombre de paris



Profit : O/U 2.5 marché

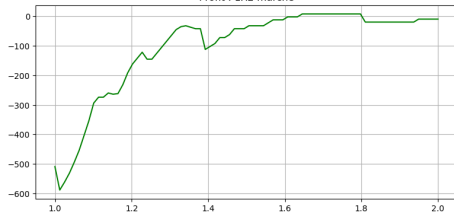


Nombre de paris

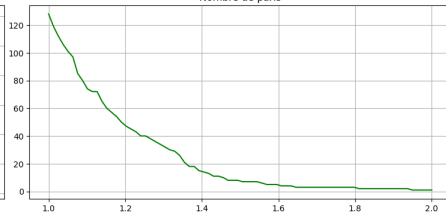


Modèle 2 :

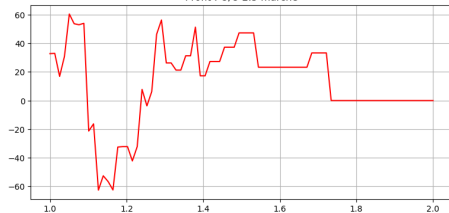
Profit : 1X2 marché



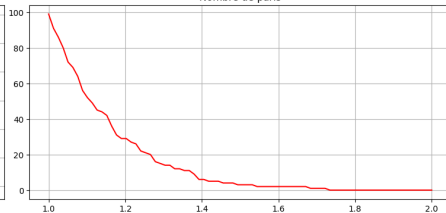
Nombre de paris



Profit : O/U 2.5 marché

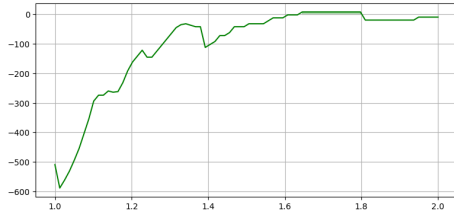


Nombre de paris

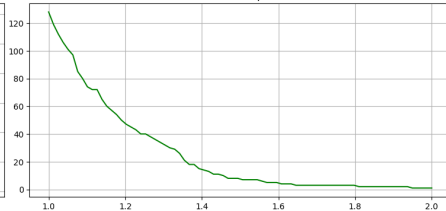


Modèle 2 :

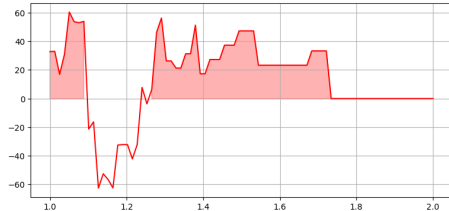
Profit : 1X2 marché



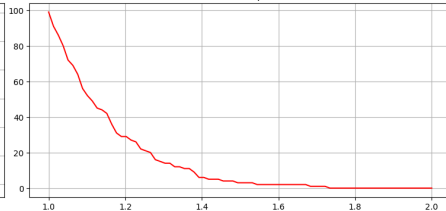
Nombre de paris



Profit : O/U 2.5 marché



Nombre de paris



Récapitulatif :

θ	Modèle 0	Modèle 1	Modèle 2
Marché 1X2 :			
Profit positif :	OUI	OUI	OUI
Profit maximal :	47.5	87.5	7.5
Nombre de paris :	9	14	3
Valeur de t qui maximise le profit :	1.62	1.55	1.64
Marché O/U 2.5 :			
Profit positif :	OUI	OUI	OUI
Profit maximal :	84.3	54.3	60.49
Nombre de paris :	10	13	72
Valeur de t qui maximise le profit :	1.46	1.329	1.05

- Les trois modèles étaient capables de bien Prévoir les résultats des matchs.

Commentaires :

- Les trois modèles étaient capables de bien Prévoir les résultats des matchs.
- Les trois modèles avait presque la même performance quant à la prédiction qui gagne le match.

- Les trois modèles étaient capables de bien Prévoir les résultats des matchs.
- Les trois modèles avait presque la même performance quant à la prédiction qui gagne le match.
- Le modèle 2 était plus performant quant à prédire si les matchs vont finir par plus ou moins de 2.5 buts.

- Les trois modèles étaient capables de bien Prévoir les résultats des matchs.
- Les trois modèles avait presque la même performance quant à la prédiction qui gagne le match.
- Le modèle 2 était plus performant quant à prédire si les matchs vont finir par plus ou moins de 2.5 buts.
- Lors des parriages, les trois modèles étaient capables d'aboutir à des profits positifs.

- Les trois modèles étaient capables de bien Prévoir les résultats des matchs.
- Les trois modèles avait presque la même performance quant à la prédiction qui gagne le match.
- Le modèle 2 était plus performant quant à prédire si les matchs vont finir par plus ou moins de 2.5 buts.
- Lors des parriages, les trois modèles étaient capables d'aboutir à des profits positifs.
- Le modèle 1 est suffisant.

Merci pour votre
attention

Pour des variables aléatoires à valeurs dans $\mathbb{N}$ on trouve

$$\begin{aligned}P(X = x, Y = y) &= P(X \leq x, Y \leq y) - P(X \leq x, Y < y) \\&= P(X \leq x, Y \leq y) - P(X \leq x, Y < y) \\&\quad + P(X < x, Y \leq y) - P(X < x, Y < y) \\&= F(x, y) - F(x, y - 1) - F(x - 1, y) + F(x - 1, y - 1)\end{aligned}$$

```

[]: from bs4 import BeautifulSoup
import os
import requests
import pandas as pd
from datetime import datetime
url='https://fbref.com/en/comps/9/schedule/Premier-League-Scores-and-Fixtures'
page=requests.get(url)
soup=BeautifulSoup(page.text)
Hometeams=[team.text for team in soup.find_all('td', class_='right') if team['data-stat'] == 'home_team'
↳and 'iz' not in team.get('class', [])]
Awayteams=[team.text for team in soup.find_all('td', class_='left') if team['data-stat'] == 'away_team'
↳and 'iz' not in team.get('class', [])]
score = [score.text for score in soup.find_all('td', class_='center') if score['data-stat'] == 'score'
↳and 'iz' not in score.get('class', [])]
Hscore=[int(score[0]) for score in score]
Ascore=[int(score[2]) for score in score]
df=pd.DataFrame({'Home':Hometeams,'Hscore':Hscore,'Ascore':Ascore,'Away':Awayteams,'date':dates})
df.to_csv(r'D:\\tipe\\tipe-2\\csv-files-2\\2023-2024.csv', encoding='utf-8', index=False)

```

```
[ ]: import pandas as pd
import numpy as np
from datetime import datetime
import multiprocessing as mp
df1=pd.read_csv("D:/tipte/tipe-2/csv-files-2/2023-2024.csv")
df2=pd.read_csv("D:/tipte/tipe-2/csv-files-2/2022-2023.csv")
df3=pd.read_csv("D:/tipte/tipe-2/csv-files-2/2021-2022.csv")
frames = [df3, df2, df1]
data=pd.concat(frames)
dates=df1['date'].unique()
data['date'] = pd.to_datetime(data['date'])
teams=np.sort(df1['Home']).unique()
n=len(teams)
data_reduced=data[(data['Home'].isin(teams)) & (data['Away'].isin(teams))]
initial_values=np.concatenate((np.random.uniform(0,1,(n)),np.random.uniform(0,-1,(n)),np.array([0.1])))
```

```
[ ]: def process_teta(date,data_reduced,initial_values,teams,n):
import pandas as pd
import numpy as np
from scipy.stats import poisson
from scipy.optimize import minimize
import os
def poisson_x(x,alpha_x,beta_y,gamma):
    lamda=np.exp(alpha_x+beta_y+gamma)
    return poisson.pmf(x,lamda)
def poisson_y(y,alpha_y,beta_x):
    lamda=np.exp(alpha_y+beta_x)
    return poisson.pmf(y,lamda)
def estimation_des_parametres(params,data):
    attack_coeff=dict(zip(teams,params[:n]))
    defence_coeff=dict(zip(teams,params[n:2*n]))
    gamma=params[-1]
    termes=[np.log(poisson_x(row.Hscore,attack_coeff[row.Home],defence_coeff[row.Away],gamma))+np.
    ↪ log(poisson_y(row.Ascore,attack_coeff[row.Away],defence_coeff[row.Home])) for row in data.itertuples()]
    somme=sum(termes)
    return -somme
```

```
[ ]: if not os.path.exists(f'D:/tipe/tipe-2/csv-folders-3/results_model_0'):
    folder_path = f'D:/tipe/tipe-2/csv-folders-3/results_model_0'
    os.makedirs(folder_path, exist_ok=True)
    if not os.path.exists(f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(date)}'):
        current_date=pd.to_datetime(date)
        data_n = data_reduced[data_reduced['date']<= current_date].copy()
        parametres=minimize(estimation_des_parametres, initial_values, args=(data_n), options={'maxiter':
↪ 100, 'disp': True},constraints = [{'type':'eq', 'fun': lambda x: sum(x[:20])-20}],bounds=[(-2, 2)])
        results=parametres.x
        os.makedirs(f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(date)}', exist_ok=True)
        df_att=pd.DataFrame({'teams':teams,'att_coef': results[:n]})
        file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(date)}/att_coef.csv'
        df_att.to_csv(file_name, encoding='utf-8', index=False)
        df_def=pd.DataFrame({'teams':teams,'def_coef': results[n:2*n]})
        file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(date)}/def_coef.csv'
        df_def.to_csv(file_name, encoding='utf-8', index=False)
        df_para=pd.DataFrame({'parameters':['gamma'],'value': results[-1]})
        file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(date)}/gamma.csv'
        df_para.to_csv(file_name, encoding='utf-8', index=False)
```

```
[ ]: if __name__ == '__main__':
    dates_list = [(date,data_reduced,initial_values,teams,n) for date in work_data(dates)][9:]
    with mp.Pool() as pool:
        pool.starmap(process_teta,dates_list)
```

```
[ ]: import pandas as pd
import numpy as np
from datetime import datetime
import multiprocessing as mp
def work_data(dates):
    date_ref=dates[0]
    weeks=[[date_ref]]
    for date in dates:
        if datetime.strptime(date, '%Y-%m-%d').date().isocalendar()[2] == datetime.strptime(date_ref,
↪ '%Y-%m-%d').date().isocalendar()[2]:
            weeks[-1].append(date)
        else :
            weeks.append([date])
            date_ref=date
    return [weeks[i][-1] for i in range(len(weeks))]
df1=pd.read_csv("D:/tipe/tipe-2/csv-files-2/2023-2024.csv")
df2=pd.read_csv("D:/tipe/tipe-2/csv-files-2/2022-2023.csv")
df3=pd.read_csv("D:/tipe/tipe-2/csv-files-2/2021-2022.csv")
frames = [df3, df2, df1]
data=pd.concat(frames)
dates=df1['date'].unique()
data['date'] = pd.to_datetime(data['date'])
teams=np.sort(df1['Home'].unique())
n=len(teams)
data_reduced=data[(data['Home'].isin(teams)) & (data['Away'].isin(teams))]
initial_values=np.concatenate((np.random.uniform(0,1,(n)),np.random.uniform(0,-1,(n)),np.array([0.1])))
```

```
[ ]: def process_teta(teta ,date,data_reduced,initial_values,teams,n):
import pandas as pd
import numpy as np
from scipy.stats import poisson
from scipy.optimize import minimize
import os
def poisson_x(x,alpha_x,beta_y,gamma):
    lamda=np.exp(alpha_x+beta_y+gamma)
    return poisson.pmf(x,lamda)
def poisson_y(y,alpha_y,beta_x):
    lamda=np.exp(alpha_y+beta_x)
    return poisson.pmf(y,lamda)
def estimation_des_parametres(params,data):
    attack_coeff=dict(zip(teams,params[:n]))
    defence_coeff=dict(zip(teams,params[n:2*n]))
    gamma=params[-1]
    termes=[(np.log(poisson_x(row.Hscore,attack_coeff[row.Home],defence_coeff[row.Away],gamma))+
               np.log(poisson_y(row.Ascore,attack_coeff[row.Away],defence_coeff[row.Home]))
               *np.exp(-row.date_diff*teta)) for row in data.itertuples()]
    somme=sum(termes)
    return -somme
```

```
[ ]: if not os.path.exists(f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}'):
    folder_path = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}'
    os.makedirs(folder_path, exist_ok=True)
if not os.path.exists(f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(date)}'):
    current_date=pd.to_datetime(date)
    data_n = data_reduced[data_reduced['date']<= current_date].copy()
    dates_r=data_n['date']
    date_diff=[(current_date - date).days for date in dates_r]
    data_n.loc[:, 'date_diff'] = date_diff
    parametres=minimize(estimation_des_parametres, initial_values, args=(data_n), options={'maxiter':
↪ 100, 'disp': True},constraints = [{'type':'eq', 'fun': lambda x: sum(x[:20])-20}],bounds=[(-2, 2)])
    results=parametres.x
    os.makedirs(f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(date)}',
↪ exist_ok=True)
    df_att=pd.DataFrame({'teams':teams,'att_coef': results[:n]})
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(date)}/
↪ att_coef.csv'
    df_att.to_csv(file_name, encoding='utf-8', index=False)
    df_def=pd.DataFrame({'teams':teams,'def_coef': results[n:2*n]})
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(date)}/
↪ def_coef.csv'
    df_def.to_csv(file_name, encoding='utf-8', index=False)
    df_para=pd.DataFrame({'parametres':['gamma'],'value': results[-1]})
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(date)}/gamma.
↪ csv'
    df_para.to_csv(file_name, encoding='utf-8', index=False)
```

```
[ ]: if __name__ == '__main__':
    dates_list = [(0.002,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:
↪ -1]+[(0.004,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:-1]+[(0.
↪ 005,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:-1]+[(0.
↪ 006,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:-1]+[(0.
↪ 007,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:-1]+[(0.
↪ 008,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:-1]+[(0.
↪ 009,date,data_reduced,initial_values,teams,n) for date in work_data(dates)][17:-1]
    with mp.Pool() as pool:
        pool.starmap(process_teta,dates_list)
```

```
[ ]: import pandas as pd
import numpy as np
from datetime import datetime
def work_data(dates):
    date_ref=dates[0]
    weeks=[[date_ref]]
    for date in dates:
        if datetime.strptime(date, '%Y-%m-%d').date().isocalendar()[2] == datetime.strptime(date_ref,
↪ '%Y-%m-%d').date().isocalendar()[2]:
            weeks[-1].append(date)
        else :
            weeks.append([date])
            date_ref=date
    return [weeks[i][-1] for i in range(len(weeks))]
df1=pd.read_csv("D:/tiipe/tiipe-2/csv-files-2/2023-2024.csv")
df2=pd.read_csv("D:/tiipe/tiipe-2/csv-files-2/2022-2023.csv")
df3=pd.read_csv("D:/tiipe/tiipe-2/csv-files-2/2021-2022.csv")
frames = [df3, df2, df1]
data=pd.concat(frames)
dates=df1['date'].unique()
data['date'] = pd.to_datetime(data['date'])
teams=np.sort(df1['Home']).unique()
n=len(teams)
data_reduced=data[(data['Home'].isin(teams)) & (data['Away'].isin(teams))]
initial_values=np.concatenate((np.random.uniform(0,1,(n)),
                                np.random.uniform(0,-1,(n)),np.array([0.1,0.1])))
```

```
[ ]: def process_teta(date,data_reduced,teta,initial_values,teams,n):
import pandas as pd
import numpy as np
from scipy.stats import poisson
from scipy.optimize import minimize
import os

def poisson_X(alpha,beta,gamma,x):
    lamda=np.exp(alpha+beta+gamma)
    m=0
    for i in range(x+1):
        m+=poisson.pmf(i,lamda)
    return m

def poisson_Y(alpha,beta,x):
    lamda=np.exp(alpha+beta)
    m=0
    for i in range(x+1):
        m+=poisson.pmf(i,lamda)
    return m

def Frank(f,g,tau):
    a=np.exp(-tau*f)-1
    b=np.exp(-tau*g)-1
    c=np.exp(-tau)-1
    return (-1/tau)*np.log(1+(a*b)/c)

def FCT(x,y,tau,alpha_x,alpha_y,beta_x,beta_y,gamma):
    return (Frank(poisson_X(alpha_x,beta_y,gamma,x),poisson_Y(alpha_y,beta_x,y),tau)
            -Frank(poisson_X(alpha_x,beta_y,gamma,x-1),poisson_Y(alpha_y,beta_x,y),tau)
            -Frank(poisson_X(alpha_x,beta_y,gamma,x),poisson_Y(alpha_y,beta_x,y-1),tau)
            +Frank(poisson_X(alpha_x,beta_y,gamma,x-1),poisson_Y(alpha_y,beta_x,y-1),tau))

def logMLE(x,y,alpha_x,alpha_y,beta_x,beta_y,tau,gamma,teta,t):
    return np.log(FCT(x,y,tau,alpha_x,alpha_y,beta_x,beta_y,gamma))*np.exp(-t*teta)

def estimation_des_parametres(params,data):
    attack_coeff=dict(zip(teams,params[:n]))
    defence_coeff=dict(zip(teams,params[n:2*n]))
    tau,gamma=params[-2:]
    termes=[logMLE(row.Hscore,row.Ascore,attack_coeff[row.Home],attack_coeff[row.Away],
                    defence_coeff[row.Home],defence_coeff[row.Away],tau,gamma,teta,
                    row.date_diff) for row in data.itertuples()]
    somme=sum(termes)
    return -somme
```

```
[ ]: if not os.path.exists(f'D:/tipe/tipe-2/csv-folders-3/results_{teta}'):
    folder_path = f'D:/tipe/tipe-2/csv-folders-3/results_{teta}'
    os.makedirs(folder_path, exist_ok=True)
if not os.path.exists(f'D:/tipe/tipe-2/csv-folders-3/results_{teta}/results_{str(date)}'):
    current_date=pd.to_datetime(date)
    data_n = data_reduced[data_reduced['date']<= current_date].copy()
    dates_r=data_n['date']
    date_diff=[(current_date - date).days for date in dates_r]
    data_n.loc[:, 'date_diff'] = date_diff
    parametres=minimize(estimation_des_parametres, initial_values, args=(data_n),
                        options={'maxiter':100, 'disp': True},
                        constraints = [{'type':'eq', 'fun': lambda x: sum(x[:20])-20}],
                        bounds=[(-2, 2)])
    results=parametres.x
    os.makedirs(f'D:/tipe/tipe-2/csv-folders-3/results_{teta}/results_{str(date)}', exist_ok=True)
    df_att=pd.DataFrame({'teams':teams,'att_coef': results[:n]})
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_{teta}/results_{str(date)}/att_coef.csv'
    df_att.to_csv(file_name, encoding='utf-8', index=False)
    df_def=pd.DataFrame({'teams':teams,'def_coef': results[n:2*n]})
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_{teta}/results_{str(date)}/def_coef.csv'
    df_def.to_csv(file_name, encoding='utf-8', index=False)
    df_para=pd.DataFrame({'parametres':['tau','gamma'],'value': results[-2:]})
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_{teta}/results_{str(date)}/tau_gamma.csv'
    df_para.to_csv(file_name, encoding='utf-8', index=False)
if __name__ == '__main__':
    dates_list = [(date,data_reduced,0.001,initial_values,teams,n) for date in work_data(dates)]
    with mp.Pool() as pool:
        pool.starmap(process_teta,dates_list)
```

```
[ ]: import pandas as pd
import numpy as np
import os
from scipy.stats import poisson
from datetime import datetime
import glob
import statistics
data=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\data.csv")
data_glob=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\data_glob.csv")
import matplotlib.pyplot as plt
```

```
[2]: def work_data(dates):
    date_ref=dates.iloc[0]
    weeks=[[date_ref]]
    for date in dates:
        if datetime.strptime(date, '%Y-%m-%d').date().isocalendar()[2] == datetime.strptime(date_ref, '%Y-%m-%d').date().isocalendar()[2]:
            weeks[-1].append(date)
        else :
            weeks.append([date])
            date_ref=date
    return [weeks[i][-1] for i in range(len(weeks))]
def what_week(date,dates):
    n=len(dates)
    for i in range(0,n) :
        if datetime.strptime(date, '%Y-%m-%d').date().isocalendar()[2] == datetime.strptime(dates[i], '%Y-%m-%d').date().isocalendar()[2]:
            return dates[i-1]
```

```
[ ]: def poisson_X(alpha,beta,gamma,x):
    lamda=np.exp(alpha+beta+gamma)
    m=0
    for i in range(x+1):
        m+=poisson.pmf(i,lamda)
    return m
def poisson_Y(alpha,beta,x):
    lamda=np.exp(alpha+beta)
    m=0
    for i in range(x+1):
        m+=poisson.pmf(i,lamda)
    return m
def Frank(f,g,tau):
    a=np.exp(-tau*f)-1
    b=np.exp(-tau*g)-1
    c=np.exp(-tau)-1
    return (-1/tau)*np.log(1+(a*b)/c)

def FCT(x,y,tau,alpha_x,alpha_y,beta_x,beta_y,gamma):
    return (Frank(poisson_X(alpha_x,beta_y,gamma,x),poisson_Y(alpha_y,beta_x,y),tau)
            -Frank(poisson_X(alpha_x,beta_y,gamma,x-1),poisson_Y(alpha_y,beta_x,y),tau)
            -Frank(poisson_X(alpha_x,beta_y,gamma,x),poisson_Y(alpha_y,beta_x,y-1),tau)
            +Frank(poisson_X(alpha_x,beta_y,gamma,x-1),poisson_Y(alpha_y,beta_x,y-1),tau))

dates=data['date'][759:]
dates_filtered=work_data(dates)
```

```
[3]: teta_values=[0,0.001,0.002,0.003,0.004,0.005,0.006,0.007,0.008,0.009]
L=[]
for teta in teta_values:
    l=0
    for row in [row for row in data_glob.itertuples()][237:]:
        date=row.date
        week=what_week(date,dates_filtered)
        file_name = f'D:/tipe/tipe-2/csv-folders-3/results_{str(teta)}/results_{str(week)}/att_coef.csv'
        att_coef=pd.read_csv(file_name)
        file_name = f'D:/tipe/tipe-2/csv-folders-3/results_{str(teta)}/results_{str(week)}/def_coef.csv'
        def_coef=pd.read_csv(file_name)
        file_name = f'D:/tipe/tipe-2/csv-folders-3/results_{str(teta)}/results_{str(week)}/tau_gamma.csv'
        tau_gamma=pd.read_csv(file_name)
        path =f"D:\\tipe\\tipe-2\\csv-folders-3\\results_{str(teta)}"
        alpha_x=att_coef.loc[att_coef['teams'] ==row.Home , 'att_coef'].values[0]
        alpha_y=att_coef.loc[att_coef['teams'] ==row.Away , 'att_coef'].values[0]
        bet_x=def_coef.loc[def_coef['teams'] ==row.Home , 'def_coef'].values[0]
        bet_y=def_coef.loc[def_coef['teams'] ==row.Away , 'def_coef'].values[0]
        tau=tau_gamma.loc[tau_gamma['parametres'] =='tau' , 'value'].values[0]
        gamma=tau_gamma.loc[tau_gamma['parametres'] =='gamma' , 'value'].values[0]
        p=np.log(FCT(row.Hscore,row.Ascore,tau,alpha_x,alpha_y,bet_x,bet_y,gamma))
        l+=p
    L.append(l)
print(L)
```

```
[-441.2465070078306, -439.92744928321645, -438.8335347046061,
-437.98847646887435, -437.40045581972674, -437.0586095492087,
-436.9369229899004, -436.99912760126136, -437.2194093176425,
-437.56699709906883]
```

```
[4]: def poisson_x(x,alpha_x,beta_y,gamma):
      lamda=np.exp(alpha_x+beta_y+gamma)
      return poisson.pmf(x,lamda)
      def poisson_y(y,alpha_y,beta_x):
        lamda=np.exp(alpha_y+beta_x)
        return poisson.pmf(y,lamda)
      teta_values=[0,0.001,0.002,0.003,0.004,0.005,0.006,0.007,0.008,0.009]
      L1=[]
      for teta in teta_values:
        l=0
        for row in [row for row in data_glob.itertuples()][237:]:
          date=row.date
          week=what_week(date,dates_filtered)
          file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(week)}/
      ↪att_coef.csv'
          att_coef=pd.read_csv(file_name)
          file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(week)}/
      ↪def_coef.csv'
          def_coef=pd.read_csv(file_name)
          file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model0_{str(teta)}/results_{str(week)}/gamma.
      ↪csv'
          gamma=pd.read_csv(file_name)
          alpha_x=att_coef.loc[att_coef['teams'] ==row.Home , 'att_coef'].values[0]
          alpha_y=att_coef.loc[att_coef['teams'] ==row.Away , 'att_coef'].values[0]
          bet_x=def_coef.loc[def_coef['teams'] ==row.Home , 'def_coef'].values[0]
          bet_y=def_coef.loc[def_coef['teams'] ==row.Away , 'def_coef'].values[0]
          gamma=gamma.loc[gamma['parametres'] =='gamma' , 'value'].values[0]
          p=np.log(poisson_x(row.Hscore,alpha_x,bet_y,gamma)*poisson_y(row.Ascore,alpha_y,bet_x))
          l+=p
        L1.append(l)
      print(L1)
```

```
[-441.12239233950567, -440.13459106269653, -439.3040605903637,
-438.69638939750604, -438.3115875411876, -438.1109771325881, -438.0352334556768,
-438.045162295011, -438.10878652605476, -438.20723436945815]
```

```
[5]: def poisson_x(x,alpha_x,beta_y,gamma):
    lamda=np.exp(alpha_x+beta_y+gamma)
    return poisson.pmf(x,lamda)
def poisson_y(y,alpha_y,beta_x):
    lamda=np.exp(alpha_y+beta_x)
    return poisson.pmf(y,lamda)

l=0
for row in [row for row in data_glob.itertuples()][237:]:
    date=row.date
    week=what_week(date,dates_filtered)
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(week)}/att_coef.csv'
    att_coef=pd.read_csv(file_name)
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(week)}/def_coef.csv'
    def_coef=pd.read_csv(file_name)
    file_name = f'D:/tipe/tipe-2/csv-folders-3/results_model_0/results_{str(week)}/gamma.csv'
    gamma=pd.read_csv(file_name)
    alpha_x=att_coef.loc[att_coef['teams'] ==row.Home , 'att_coef'].values[0]
    alpha_y=att_coef.loc[att_coef['teams'] ==row.Away , 'att_coef'].values[0]
    bet_x=def_coef.loc[def_coef['teams'] ==row.Home , 'def_coef'].values[0]
    bet_y=def_coef.loc[def_coef['teams'] ==row.Away , 'def_coef'].values[0]
    gamma=gamma.loc[gamma['parametres'] =='gamma' , 'value'].values[0]
    p=np.log(poisson_x(row.Hscore,alpha_x,bet_y,gamma)*poisson_y(row.Ascore,alpha_y,bet_x))
    l+=p
print(l)
```

-441.1201533485595

```
[2]: model_2=pd.read_csv("D:/tipe/tipe-2/csv-files-3/data_glob_0.002.csv")
      model_1=pd.read_csv("D:/tipe/tipe-2/csv-files-3/data_glob_model0_0.005.csv")
      model_0=pd.read_csv("D:/tipe/tipe-2/csv-files-3/data_glob_model0_0.csv")
```

```
[3]: def bet_1X2(t,row):
      proba_list=[row.proba_win,row.proba_draw,row.proba_lose]
      X2_book=[row.Odds_1,row.Odds_X,row.Odds_2]
      E=[(proba_list[i]*X2_book[i]) for i in range(3)]
      maximum=max(E)
      i=E.index(maximum)
      O=X2_book[i]
      if maximum>t:
          if i==0:
              if row.Hscore>row.Ascore:
                  return (10*(O-1),1)
              else :
                  return (-10,1)
          elif i==1:
              if row.Hscore==row.Ascore:
                  return (10*(O-1),1)
              else :
                  return (-10,1)
          else:
              if row.Hscore<row.Ascore:
                  return (10*(O-1),1)
              else :
                  return (-10,1)
      else:
          return (0,0)
```

```
[4]: def bet_OU(t,row):
    proba_list=[row.proba_0,row.proba_U]
    ou_book=[row.O,row.U]
    E=[(proba_list[i]*ou_book[i]) for i in range(2)]
    maximum=max(E)
    i=E.index(maximum)
    O=ou_book[i]
    if maximum>t:
        if i==0:
            if row.Hscore+row.Ascore>=3:
                return (10*(O-1),1)
            else :
                return (-10,1)
        else:
            if row.Hscore+row.Ascore<=2:
                return (10*(O-1),1)
            else :
                return (-10,1)
    else:
        return (0,0)
```

```
[5]: def X_profit(t,data_all):
    profit=0
    c=0
    for row in [row for row in data_all.itertuples()][237:]:
        profit+=bet_1X2(t,row)[0]
        c+=bet_1X2(t,row)[1]
    return profit,c
def OU_profit(t,data_all):
    c=0
    profit=0
    for row in [row for row in data_all.itertuples()][237:]:
        profit+=bet_OU(t,row)[0]
        c+=bet_OU(t,row)[1]
    return profit,c
```

```
[8]: t=np.linspace(1,2,80)
def results(data):
    X_p=[]
    OU_p=[]
    nb_bets_1X2=[]
    nb_bets_OU=[]
    for i in t:
        _1X2_=X_profit(i,data)
        _OU_=OU_profit(i,data)
        X_p.append(_1X2_[0])
        OU_p.append(_OU_[0])
        nb_bets_1X2.append(_1X2_[1])
        nb_bets_OU.append(_OU_[1])
    return (X_p,OU_p,nb_bets_1X2,nb_bets_OU)
```

```
[9]: print(results(model_0)[0])
print(results(model_0)[1])
print(results(model_0)[2])
print(results(model_0)[3])
```

```
[-362.3, -332.3, -311.70000000000005, -286.70000000000005, -269.5, -251.7,
-266.20000000000005, -290.5, -260.5, -234.9, -211.5, -191.5, -141.5, -111.5,
-101.5, -69.5, -59.5, -92.0, -82.0, -97.5, -77.5, -67.5, -157.5, -137.5, -127.5,
-107.5, -87.5, -67.5, -57.5, -17.5, -17.5, 2.5, 12.5, 32.5, 32.5, 32.5,
32.5, 7.5, 7.5, 7.5, 7.5, 17.5, 17.5, 17.5, 17.5, 17.5, 27.5, 37.5, 47.5, -42.5,
-32.5, -32.5, -32.5, -22.5, -12.5, -12.5, -2.5, -2.5, -2.5, -2.5, -2.5, -2.5,
-2.5, -2.5, -2.5, -2.5, -2.5, -2.5, -2.5, -2.5, -2.5, -2.5, -30, -30, -30, -30,
-20, -20, -20]
[-70.10000000000001, -56.40000000000002, -60.7, -72.90000000000002,
-60.900000000000006, -82.10000000000004, -29.200000000000017,
-30.500000000000014, -20.500000000000014, 0.4999999999999929,
13.499999999999993, -13.500000000000014, -49.800000000000001,
-29.500000000000007, -71.60000000000001, -44.60000000000001, -34.6,
-14.600000000000001, -14.600000000000001, -39.6, -41.600000000000001,
-41.600000000000001, -45.100000000000001, -59.100000000000001, -59.100000000000001,
-29.1, 10.899999999999999, 8.899999999999999, 8.299999999999997,
28.299999999999997, 14.299999999999997, 34.3, 64.3, 74.3, 74.3, 74.3, 74.3,
84.3, 59.3, 59.3, 29.299999999999997, 17.299999999999997, -6.700000000000003,
13.299999999999997, 13.299999999999997, 13.299999999999997, 33.3, 33.3, 33.3,
33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3,
```

```

33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 33.3, 0, 0, 0, 0,
0, 0]
[117, 114, 106, 101, 95, 92, 89, 80, 77, 73, 66, 64, 59, 56, 55, 50, 49, 46, 45,
42, 40, 39, 33, 31, 30, 28, 26, 24, 23, 19, 19, 17, 16, 14, 14, 14, 14, 13,
13, 13, 13, 12, 12, 12, 12, 12, 11, 10, 9, 8, 7, 7, 7, 6, 5, 5, 4, 4, 4, 4, 4,
4, 4, 4, 4, 4, 4, 4, 4, 4, 3, 3, 3, 3, 2, 2, 2]
[118, 113, 108, 104, 98, 94, 85, 80, 79, 73, 69, 65, 62, 58, 52, 47, 46, 44, 44,
42, 40, 37, 35, 33, 33, 30, 26, 23, 21, 19, 17, 15, 12, 11, 11, 11, 11, 10, 9,
9, 8, 6, 5, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]

```

```

[10]: print(results(model_1)[0])
      print(results(model_1)[1])
      print(results(model_1)[2])
      print(results(model_1)[3])

```

```

[-318.6, -309.1, -279.3, -216.8, -230.10000000000002, -223.1, -179.1, -166.1,
-146.1, -105.19999999999999, -100.5, -80.5, -60.5, -93.0, -173.0, -167.0,
-172.0, -162.0, -159.0, -139.0, -119.0, -99.0, -97.5, -57.5, -27.5, -17.5, -7.5,
-7.5, 2.5, 22.5, 32.5, 42.5, 62.5, 62.5, 62.5, 72.5, 72.5, 57.5, 67.5, 67.5,
67.5, 77.5, 77.5, 77.5, 87.5, 87.5, 62.5, 72.5, 82.5, 47.5, 47.5, 47.5, 47.5,
67.5, 77.5, 77.5, 77.5, 77.5, 77.5, 87.5, -2.5, -2.5, -2.5, 7.5, 7.5, 7.5, 7.5,
-20, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10, -10]
[-104.7, -107.3, -104.20000000000003, -41.3, -64.2, -49.900000000000002,
5.5999999999999994, 7.4999999999999993, -8.800000000000004, -42.8,
-22.800000000000004, -37.6, -36.1, -36.1, -43.10000000000001, -23.1,
4.899999999999999, -1.1000000000000014, -15.100000000000001, -5.100000000000001,
-15.100000000000001, 4.899999999999999, 24.9, 34.9, 44.9, 44.9, 54.3, 54.3,
24.299999999999997, 24.299999999999997, 24.299999999999997, 34.3, 44.3, 54.3,
54.3, 54.3, 29.299999999999997, 29.299999999999997, 27.299999999999997,
3.299999999999997, 3.299999999999997, 13.299999999999997, 23.299999999999997,
23.299999999999997, 23.299999999999997, 23.299999999999997, 23.299999999999997,
23.299999999999997, 23.299999999999997, 23.299999999999997, 23.299999999999997,
23.299999999999997, 23.299999999999997, 23.299999999999997, 23.299999999999997,
-10, -10, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[119, 113, 106, 96, 93, 85, 77, 73, 71, 65, 63, 61, 59, 57, 53, 51, 48, 47, 44,
42, 40, 38, 36, 32, 29, 28, 27, 26, 24, 23, 22, 20, 20, 20, 19, 19, 17, 16,
16, 16, 15, 15, 15, 14, 14, 13, 12, 11, 9, 9, 9, 9, 7, 6, 6, 6, 6, 5, 4, 4,
4, 3, 3, 3, 3, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
[120, 115, 109, 96, 87, 76, 67, 61, 52, 50, 48, 45, 40, 40, 38, 36, 31, 26, 24,

```

```
23, 22, 20, 18, 17, 16, 16, 13, 13, 12, 12, 12, 11, 10, 9, 9, 9, 8, 8, 5, 4, 4,
3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
[11]: print(results(model_2)[0])
      print(results(model_2)[1])
      print(results(model_2)[2])
      print(results(model_2)[3])
```

```
[-508.8, -588.2, -561.1, -530.8, -493.8, -453.8, -404.1, -354.1, -294.1, -274.1,
-274.1, -259.9, -264.0, -262.0, -232.0, -192.0, -162.0, -142.0, -122.0, -145.5,
-145.5, -125.5, -105.5, -85.5, -65.5, -45.5, -35.5, -32.5, -37.5, -42.5, -42.5,
-112.5, -102.5, -92.5, -72.5, -72.5, -62.5, -42.5, -42.5, -42.5, -32.5, -32.5,
-32.5, -32.5, -22.5, -12.5, -12.5, -12.5, -2.5, -2.5, -2.5, 7.5, 7.5, 7.5, 7.5,
7.5, 7.5, 7.5, 7.5, 7.5, 7.5, 7.5, 7.5, -20, -20, -20, -20, -20, -20, -20,
-20, -20, -20, -20, -10, -10, -10, -10, -10]
[32.8, 32.99999999999999, 16.89999999999999, 30.899999999999984,
60.499999999999986, 53.7, 52.999999999999986, 54.0, -21.299999999999997,
-16.299999999999997, -62.600000000000001, -52.600000000000001, -56.600000000000001,
-62.5, -32.5, -32.2, -32.2, -42.2, -32.2, 7.799999999999997, -3.700000000000003,
6.299999999999997, 46.3, 56.3, 26.299999999999997, 26.299999999999997,
21.299999999999997, 21.299999999999997, 31.299999999999997, 31.299999999999997,
51.3, 17.299999999999997, 17.299999999999997, 27.299999999999997,
27.299999999999997, 27.299999999999997, 37.3, 37.3, 37.3, 47.3, 47.3,
47.3, 23.299999999999997, 23.299999999999997, 23.299999999999997,
23.299999999999997, 23.299999999999997, 23.299999999999997, 23.299999999999997,
23.299999999999997, 23.299999999999997, 23.299999999999997, 23.299999999999997,
33.3, 33.3, 33.3, 33.3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0]
[128, 119, 112, 106, 101, 97, 85, 80, 74, 72, 72, 65, 60, 57, 54, 50, 47, 45,
43, 40, 40, 38, 36, 34, 32, 30, 29, 26, 21, 18, 18, 15, 14, 13, 11, 11, 10, 8,
8, 8, 7, 7, 7, 7, 6, 5, 5, 5, 4, 4, 4, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 2,
2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1]
[99, 91, 86, 80, 72, 69, 64, 56, 52, 49, 45, 44, 42, 36, 31, 29, 29, 27, 26, 22,
21, 20, 16, 15, 14, 14, 12, 12, 11, 11, 9, 6, 6, 5, 5, 5, 4, 4, 4, 3, 3, 3, 3,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
[12]: def plot(data,bool):
    X_p=results(data)[0]
    nb_bets_1X2=results(data)[1]
    OU_p=results(data)[2]
    nb_bets_0U=results(data)[3]
    fig1, axis = plt.subplots(2, 2)
    axis[0,1].plot(t,nb_bets_1X2,color='g')
    axis[0,1].set_title('Nombre de paris')
    axis[0,0].plot(t,X_p,color='g')
    axis[0,0].set_title('Profit : 1X2 marché')
    if bool:axis[0,0].fill_between(t, X_p, color='green',where=(np.array(X_p) >= 0), alpha=0.3)
    axis[0,0].grid()
    axis[0,1].grid()
    axis[1,1].plot(t,nb_bets_0U,color='r')
    axis[1,1].set_title('Nombre de paris')
    axis[1,0].plot(t,OU_p,color='r')
    axis[1,0].set_title('Profit : 0/U 2.5 marché')
    if bool:axis[1,0].fill_between(t, OU_p, color='red',where=(np.array(OU_p) >= 0), alpha=0.3)
    axis[1,0].grid()
    axis[1,1].grid()
    plt.subplots_adjust(left=0.036, right=0.99, top=0.966, bottom=0.04, wspace=0.06, hspace=0.162)
    plt.show()
```

```
[23]: import pandas as pd
import statistics
import numpy as np
import os
import glob
```

```
[24]: df1=pd.read_csv("D:/tipe/tipe-2/csv-files-2/2023-2024.csv")
teams=np.sort(df1['Home'].unique())
print(teams)
path="D:\\tipe\\tipe-2\\csv-folders-3\\results_0.002"
A=[]
D=[]
for team in teams :
    deff=[]
    att=[]
    for filename in list(glob.glob(f"{path}/*"))[24:]:
        att_coeff=pd.read_csv(os.path.join(path,filename,f"att_coeff.csv"))
        def_coeff=pd.read_csv(os.path.join(path,filename,f"def_coeff.csv"))
        att.append(att_coeff.loc[att_coeff['teams'] ==team , 'att_coef'].values[0])
        deff.append(def_coeff.loc[def_coeff['teams'] ==team , 'def_coef'].values[0])
    D.append(-statistics.mean(deff))
    A.append(statistics.mean(att))
print(A)
print(D)
A_sorted=sorted(A)
print([20-A_sorted.index(el) for el in A])
D_sorted=sorted(D)
print([20-D_sorted.index(el) for el in D])
```

['Arsenal' 'Aston Villa' 'Bournemouth' 'Brentford' 'Brighton' 'Burnley'
 'Chelsea' 'Crystal Palace' 'Everton' 'Fulham' 'Liverpool' 'Luton Town'
 'Manchester City' 'Manchester Utd' 'Newcastle Utd' 'Nott'ham Forest'
 'Sheffield Utd' 'Tottenham' 'West Ham' 'Wolves']

[1.4365178584273588, 1.1698106461053566, 0.8326651819585864, 0.9813983782695586,
 1.0972645734438096, 0.6508797928605806, 1.0640283997171966, 0.7584660794738725,
 0.6315369123324968, 0.9480057436352467, 1.3673689940608171, 0.9813484862444618,
 1.465633748919382, 1.015973388545177, 1.283387301846496, 0.7848796753018072,
 0.566819353072956, 1.2370486652217207, 0.9561122090116483, 0.7708546115514695]
 [1.145751947019032, 0.8324570369718264, 0.5657484454309746, 0.7427599413475912,
 0.7566072928510371, 0.5254914793564627, 0.7842681489726844, 0.7459047244173931,
 0.8010453840461819, 0.7588407484037206, 1.1349897140978897, 0.4111073142637537,
 1.2373789450024038, 0.8467692848001881, 0.7944774060070892, 0.6090003280521005,
 0.20070350311692925, 0.736770542008351, 0.6470933603679144, 0.7717448449892563]
 [2, 6, 14, 10, 7, 18, 8, 17, 19, 13, 3, 11, 1, 9, 4, 15, 20, 5, 12, 16]
 [2, 5, 17, 13, 11, 18, 8, 12, 6, 10, 3, 19, 1, 4, 7, 16, 20, 14, 15, 9]

```
[2]: teta_values=[0,0.001,0.002,0.003,0.004,0.005,0.006,0.007,0.008,0.009]
s_values=[]
for teta in teta_values:
    data=pd.read_csv(f"D:/tipe/tipe-2/csv-files-3/data_glob_model0_{str(teta)}.csv")
    s=0
    for row in [row for row in data.itertuples()][196:]:
        if row.Hscore>row.Ascore:
            s+=np.log(row.proba_win)
        elif row.Hscore<row.Ascore:
            s+=np.log(row.proba_lose)
        else :
            s+=np.log(row.proba_draw)
        if row.Hscore+row.Ascore>2.5:
            s+=np.log(row.proba_0)
        else:
            s+=np.log(row.proba_U)
    s_values.append(s)
print(s_values)
```

```
[-285.5305978102268, -285.107555415836, -284.7789710352653, -284.55490183935586,
-284.43477849560156, -284.4017318996272, -284.42944352975627,
-284.49601118100884, -284.58710779246746, -284.6937751143062]
```

```
[ ]: plt.plot(teta_values,s_values)
plt.scatter(teta_values , s_values)
max_index = s_values.index(max(s_values))
max_theta = teta_values[max_index]
max_tilde_ell = s_values[max_index]
plt.scatter(teta_values[max_index], s_values[max_index], color='r', zorder=5)
plt.axvline(x=max_theta, color='r', linestyle='--')
plt.axhline(y=max_tilde_ell, color='r', linestyle='--')
plt.text(max_theta, plt.ylim()[1], f'{max_theta}', ha='center', va='bottom', color='r')
plt.xlabel('$\\theta$', fontsize=14)
plt.ylabel('$S$', rotation=0, labelpad=20, fontsize=14)
plt.grid()
```

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
import glob
import os
import statistics
```

```
[2]: teta=0.002
data=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2023-2024.csv")
teams=[team for team in np.sort(data['Home'].unique())]
print(teams)
```

['Arsenal', 'Aston Villa', 'Bournemouth', 'Brentford', 'Brighton', 'Burnley', 'Chelsea', 'Crystal Palace', 'Everton', 'Fulham', 'Liverpool', 'Luton Town', 'Manchester City', 'Manchester Utd', 'Newcastle Utd', 'Nott'ham Forest', 'Sheffield Utd', 'Tottenham', 'West Ham', 'Wolves']

```
[3]: def mondays(dates):
    mondays=[]
    date=dates[0]
    date_ref=datetime.strptime(date, '%Y-%m-%d')
    monday= date_ref- timedelta(days =date_ref.weekday())
    mondays=[monday]
    for date in dates[1:]:
        _date_=datetime.strptime(date, '%Y-%m-%d')
        if _date_.date().isocalendar()[2] != date_ref.date().isocalendar()[2]:
            monday=_date_- timedelta(days = _date_.weekday())
            mondays.append(monday)
            date_ref=_date_
    return mondays
```

```
[4]: dates=(data['date'].unique())
mondays=mondays(dates)[24:]
tau_values=[]
gamma_values=[]
path=f"D:\\tipe\\tipe-2\\csv-folders-3\\results_{str(teta)}"
for filename in [filename for filename in glob.glob(f"{path}/*")] [24:-1]:
    tau_and_gamma=pd.read_csv(os.path.join(path,filename,f"tau_gamma.csv"))
    tau_values.append(tau_and_gamma['value'].iloc[0])
    gamma_values.append(tau_and_gamma['value'].iloc[1])
tau=statistics.mean(tau_values)
gamma=statistics.mean(gamma_values)
print(tau_values)
print(gamma_values)
print(tau)
print(gamma)
```

```
[-0.3154331257229579, -0.4307344607620098, -0.3764916335033978,
-0.3657683579044718, -0.4129098916264848, -0.4044311065536965,
-0.3026969175914689, -0.1254059850633552, -0.1947664915808598,
-0.2462051386270526, -0.3555564818365533, -0.3153422309505927,
-0.3395663303449167]
[0.2581766032420437, 0.2285089872643308, 0.2326715332600431, 0.2388412722044078,
0.2222276473798277, 0.232342952904192, 0.2317036814250463, 0.2312774707156143,
0.2275326730442139, 0.227827089887933, 0.2400087010343616, 0.24934951109263,
0.2446688645000956]
-0.32194678092829365
0.23577976830421074
```

```
[ ]: fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 6))
ax1.plot(mondays,tau_values)
ax1.set_title('$\kappa$')
ax1.set_ylim(-1, 0)
ax2.plot(mondays,gamma_values)
ax2.set_title('$\gamma$')
ax2.set_ylim(0, 1)
ax1.axhline(y=tau, color='r', linestyle=':')
ax2.axhline(y=gamma, color='r', linestyle=':')
ax1.text(plt.xlim()[0], tau, s=f'{tau:.2f}', ha='right', va='bottom', color='r')
ax2.text(plt.xlim()[0], gamma, s=f'{gamma:.2f}', ha='right', va='bottom', color='r')
plt.tight_layout()
plt.show()
```

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
[2]: teta_values=[0,0.001,0.002,0.003,0.004,0.005,0.006,0.007,0.008,0.009]
s_values=[]
for teta in teta_values:
    data=pd.read_csv(f"D:/tipe/tipe-2/csv-files-3/data_glob_{str(teta)}.csv")
    s=0
    for row in [row for row in data.itertuples()][196:]:
        if row.Hscore>row.Ascore:
            s+=np.log(row.proba_win)
        elif row.Hscore<row.Ascore:
            s+=np.log(row.proba_lose)
        else :
            s+=np.log(row.proba_draw)
        if row.Hscore+row.Ascore>2.5:
            s+=np.log(row.proba_0)
        else:
            s+=np.log(row.proba_U)
    s_values.append(s)
print(s_values)
```

```
[-286.0239907708632, -285.370812743845, -285.07479813270317, -285.0860597292754,
-285.3321452459457, -285.7515508806224, -286.30006649581156, -286.9345961676073,
-287.63014325399337, -288.3684033640976]
```

```
[ ]: plt.plot(teta_values,s_values)
plt.scatter(teta_values , s_values)
max_index = s_values.index(max(s_values))
max_theta = teta_values[max_index]
max_tilde_ell = s_values[max_index]
plt.scatter(teta_values[max_index], s_values[max_index], color='r', zorder=5)
plt.axvline(x=max_theta, color='r', linestyle='--')
plt.axhline(y=max_tilde_ell, color='r', linestyle='--')
plt.text(max_theta, plt.ylim()[1], f'{max_theta}', ha='center', va='bottom', color='r')
plt.xlabel('$\\theta$', fontsize=14)
plt.ylabel('$S$', rotation=0, labelpad=20, fontsize=14)
plt.grid()
```

```
[145]: import pandas as pd
teta=0.006
model_2=pd.read_csv("D:/tipe/tipe-2/csv-files-3/data_glob_0.002.csv")
model_1=pd.read_csv("D:/tipe/tipe-2/csv-files-3/data_glob_model0_0.005.csv")
model_0=pd.read_csv("D:/tipe/tipe-2/csv-files-3/data_glob_model0_0.csv")
L=list(model_0.itertuples())[237:]
N=len(L)
print(N)
```

143

```
[146]: def prediction_1(model):
    w,d,l=0,0,0
    w_r,d_r,l_r=0,0,0
    for row in list(model.itertuples())[237:]:
        L=[row.proba_win,row.proba_draw,row.proba_lose]
        maximum=max(L)
        i_max=L.index(maximum)
        if row.Hscore>row.Ascore:
            w_r+=1
            if i_max==0:
                w+=1
        elif row.Hscore==row.Ascore:
            d_r+=1
            if i_max==1:
                d+=1
        else:
            l_r+=1
            if i_max==2:
                l+=1
    return (w_r,w),(d_r,d),(l_r,l)

print(prediction_1(model_0),prediction_1(model_1),prediction_1(model_2))
```

((65, 55), (34, 0), (44, 29)) ((65, 49), (34, 0), (44, 34)) ((65, 53), (34, 0), (44, 30))

```
[147]: def prediction_2(model):
        o,u=0,0
        o_r,u_r=0,0
        for row in list(model.itertuples())[237:]:
            L=[row.proba_0,row.proba_U]
            maximum=max(L)
            i_max=L.index(maximum)
            if row.Hscore+row.Ascore>2.5:
                o_r+=1
                if i_max==0:
                    o+=1
            else:
                u_r+=1
                if i_max==1:
                    u+=1
        return (o_r,o),(u_r,u)

print(prediction_2(model_0),prediction_2(model_1),prediction_2(model_2))
```

((92, 71), (51, 15)) ((92, 76), (51, 12)) ((92, 81), (51, 11))

```
[ ]: import scipy.stats as stats
import pandas as pd
import os
from itertools import product
import glob
import statistics
from scipy.integrate import quad
import numpy as np
from scipy import stats
data=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2023-2024.csv")
```

```
[2]: data=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\data.csv")
df1=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2020-2021.csv")
df2=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2019-2020.csv")
df3=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2018-2019.csv")
df4=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2017-2018.csv")
df5=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2018-2019.csv")
df6=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2017-2018.csv")
df7=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2016-2017.csv")
df8=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2015-2016.csv")
df9=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2014-2015.csv")
df10=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2013-2014.csv")
df11=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2012-2013.csv")
df12=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2011-2012.csv")
df13=pd.read_csv("D:\\tipe\\tipe-2\\csv-files-2\\2010-2011.csv")

data_tot=pd.concat([data,df1,df2,df3,df4,df5,df6,df7,df8,df9,df10,df11,df12,df13])
```

```
[3]: teams=np.sort(data['Home']).unique()
cartesian_product = [(x, y) for x, y in product(teams, teams) if x != y]
Tau=[]
for couple in cartesian_product:
    Home=couple[0]
    Away=couple[1]
    matches=data_tot[(data_tot['Home'] == Home) & (data_tot['Away'] == Away)]
    if len(matches)>=10:
        x=[]
        y=[]
        for idx, match in matches.iterrows():
            home_score = match['Hscore']
            away_score = match['Ascore']
            x.append( home_score)
            y.append( away_score)
        tau=stats.kendalltau(y,x)[0]
        Tau.append(tau)
print(Tau)
t=statistics.mean(Tau)
print(t)
```

```
[-0.724060106999932, -0.34020690871988585, -0.07801410431448313,
-0.07027283689263064, 0.2782823699789073, 0.6388855077784064,
0.14509525002200235, -0.20999912500546872, -0.03392484547739255,
0.34236839400873037, 0.4070135768850545, 0.4267459067940983,
0.01327329683004747, 0.26600795837367097, 0.04883862905055043,
-0.227507877596645, 0.07692307692307691, 0.23537960143467365,
-0.07233642332556182, -0.21749089913081476, -0.4424923253510505,
-0.08333333333333333, 0.4052204492365539, -0.2909286827258562,
0.24242424242424246, -0.1743714581157289, -0.09416472308615167,
-0.16666666666666669, -0.18569533817705186, -0.029526929767874632,
-0.3766588923446067, 0.2779322274854458, -0.45014913660634365,
-0.18641092980036, -0.2527553506359416, 0.08343982964941768,
-0.0576390417704235, 0.15187229019027831, -0.06254399171545343,
0.36844146546660206, 0.10475656017578483, 0.35201126454070086,
-0.2538978964487733, -0.29294793715852024, -0.08337966824563375,
-0.5963765795456273, 0.14043735340401717, 0.676210153506926,
-0.2655425022718972, 0.125, 0.5303497930706049, 0.29327786272050055,
-0.021739130434782608, -0.3131121455425747, -0.151247535495505,
0.32479818847689496, 0.15556515863952655, 0.0, -0.06189844605901729,
0.1330039791868355, -0.03671115491071762, 0.2804963567266072,
```

0.45868247229386294, -0.17049858486761835, -0.4838709677419355,
-0.16666666666666663, -0.06726727939963124, 0.34866692910423897,
-0.32686022523030667, -0.18392019594868114, -0.19289712886816485,
-0.18181818181818182, -0.5521576303742327, -0.40009880202694836,
0.10090091909944687, 0.47559486560567094, -0.34222378222022665,
0.16070147520167405, -0.2891776156271279, -0.3174510888285778,
-0.4305308489096176, -0.1882483221340047, 0.3065696697424829,
0.022523150916413784, 0.05000694589153869, -0.09191759833942816, 0.0,
0.3918358757635058, 0.15529100683443642, -0.10846522890932808,
-0.35416666666666667, 0.3224013838831172, 0.0, -0.01056383870062615,
-0.29356828197599305, -0.5807632069125022, -0.29892826582327087,
-0.4874174366759539, -0.16496718065249089, 0.3189239823010603,
-0.5918556023193138, 0.27602802678696664, -0.1424507205745474,
0.3017487364298118, 0.03126526997403612, 0.06400204809830924,
0.5614367170168085, 0.023918243661746993, -0.12555296411486888,
0.23274572326112652, -0.21817496353345528, 0.01180480411624714,
-0.15877683720748892, -0.011299615367639217, -0.19653507529013656,
0.35164043693869096, 0.3737412737209254, -0.2486868143118438, 0.0,
0.10527227119356303, -0.12524485821702988, -0.43133109281375365,
-0.07610194341477615, 0.13294393474083765, 0.06071767407189163,
-0.04824883365508533, 0.20989253928803775, -0.3119251469460218,
-0.19569842191603268, -0.47925676979305964, -0.1733487427953661,
-0.18488047749952394, -0.3593026031853063, -0.38636363636363635,
-0.1851946331679053, -0.539157389535459, 0.15519548261264562, 0.0,
0.7964913660803873, -0.15734745152973198, 0.2575510574037919,
0.6499423302067071, 0.04679082708957309, -0.4828045495852676,
-0.035355339059327376, -0.02083446370599543, 0.4834937784152281,
0.011531640100361062, 0.42263985063512266, -0.41945184422536536,
-0.10810968751610266, -0.17778875273088748, -0.15069907169657754,
0.38122128787578263, 0.08552499374980313, 0.11952286093343936,
0.26681493838602804, -0.055577001437672836, 0.2138407073063167,
0.169811320754717, -0.06640158940746631, -0.050141221056390024,
-0.3831835603823421, -0.3894521534129447, 0.08434347122457897,
0.030685820596610784, -0.40368671387966554, 0.3150980590019348]
-0.022707509400888337

```
[4]: def tau(path):
    tau_values=[]
    for filename in glob.glob(f"{path}/*"):
        tau_and_gamma=pd.read_csv(os.path.join(path,filename,f"tau_gamma.csv"))
        tau_values.append(tau_and_gamma['value'].iloc[0])
    return statistics.mean(tau_values)
path="D:\\tipe\\tipe-2\\csv-folders-3\\results_0.002"
print(tau(path))
```

-0.3412214341766806

```
[5]: def kendall(tau):
    def debye_function(x, n):
        def integrand(t):
            return (t**n) / (np.exp(t) - 1)

        integral, _ = quad(integrand, 0, x)
        return (n / x**n) * integral
    return(1-4*(1/tau)*(1-debye_function(tau,1)))
print(kendall(-0.32))
```

-0.03551920994663926